

ERRORS AND EXCEPTION HANDLING IN PYTHON

What is an Error?

- An error is a problem in a program that stops the execution.
- ERRORS can happen due to:
 - wrong code
 - unexpected input
 - External factors (like missing files, network issues)

Two Types of Errors in Python

TYPE	Meaning
Syntax errors	Mistakes in the code structure (missing, colon, brackets, etc)
Exceptions	Errors that happen during execution (e.g., divide by zero)

Example of Syntax Error

```
# Missing colon  
if True  
    Print ("Hello")
```

output

syntax error: expected ':'

Example of Runtime Exception

```
a = 10  
b = 0  
Print (a/b)
```

output

Zero Division Error: division by zero

Common Exception Types in Python.

Here's a table of frequently occurring exceptions you might see

Exception name	When it happens	Example code
Zero Division Error	wrong data type used	<code>"2" + 2</code>
Value Error	correct type but wrong value	<code>int("abc")</code>
Index Error	List index out of range	<code>my_list[10]</code>
Key Error	Accessing missing key in dictionary	<code>my_dict['not-a-key']</code>
Attribute Error	Accessing non-existent attribute or method	<code>s.append(2)</code>
Import Error	module not found	<code>Import Something</code>
Name Error	using variable not defined	<code>Print(x) (when x not declared)</code>
File not found Error	Trying to open a file that doesn't exist	<code>open("missing.txt")</code>

What is Exception Handling?

Exception Handling is a way to protect your program from crashing when an error occurs.

Python uses:

- try
- except
- else
- finally

Basic structure

```
try:  
    # code that may raise an exception  
except someError:  
    # what to do if error happens  
else:  
    # Run if no error  
finally:  
    # Always run (cleanup, close file, etc)
```

Examples of Exception Handling

Example 1: catching Division by Zero

```
try:  
    a = int(input("Enter a number: "))  
    print(10/a)  
except ZeroDivisionError:  
    print("You can't divide by zero!")
```

Example 2: Handling multiple Exceptions

```
try:  
    num = int(input("Enter a number: "))  
    result = 10/num  
    print("Result:", result)  
except ZeroDivisionError:  
    print("cannot divide by zero")  
except ValueError:  
    print("Please enter a valid number")
```

Example 3: Using finally

```
try:  
    file = open("myfile.txt", "r")  
    print(file.read())  
except fileNotFoundError:  
    print("File not found")  
finally:  
    print("closing file.. (even if error occurred)")
```


Best practice: catch specific exceptions First
Avoid catching everything. Using just `except:` unless absolutely necessary

Good:
`except ValueError:`

Avoid
`except:`

Python Imports, Libraries, modules, and packages

What is a module in Python? functions, class

A module is simply a .py file containing or variables

we can import it and use its code in another file.

Example: math module (built-in)

```
import math  
print(math.sqrt(25))
```

output: 5.0

math is a built-in module. we can use its functions like `sqrt()`.

creating your own module

Let's say we have a file greeing.py

```
# greeing.py
```

```
def namaskara(name):  
    print(f"namaskara {name}")
```

```
def goodbye(name):  
    print(f"Goodbye {name}")
```


Now use it in another file:

```
# main.py
import greetings
greeting.namekare("Ravi")
greeting.goodbye("meend")
```

Import variations

Syntax	Use case
import module	General import - use like module.function()
import module as alias	Shorten name, e.g. import numpy as np.
from module import function	Import specific function only
from module import *	Import everything (! Avoid in large projects)

Example

```
from math import sqrt
print(sqrt(36)) # no need to write math.sqrt()
```

What is a Package?

- A package is a folder containing multiple modules and an optional `__init__.py` file.
- Helps organize large projects into folders.

Example Project Structure

```
School/
├── init.py
├── student.py
└── teachers.py
```